Reporte Web y Móvil — v1.0

Proyecto: YURA — plataforma web (`yura-platform`) y app Android (`yuraapp`)

Fecha: 18-10-2026 · **Estado:** plan aprobado por el dueño del proyecto; ejecución por fases.

---

1. Resumen

La plataforma web (Laravel 12, API con Sanctum) ya permite iniciar sesión por API, pero **no tiene una API pensada para una app**: el cuaderno, las notas y el panel del padre existen solo como pantallas web. Al revisar el código para diseñar la app aparecieron **7 fallas** en la web que hay que corregir antes, porque la app las heredaría o las empeoraría (acceso de familiares dados de baja, datos de más en las respuestas, tokens eternos).

La app Android ya tiene su base: compila, tiene ingreso con token y la lista de hijos, y 2 pruebas unitarias pasan. Falta el detalle de cada hijo, que depende de una ruta nueva en la web.

2. Estado actual

App móvil (`C:\Users\Franz\AndroidStudioProjects\yuraapp`)

Elemento	Estado
Paquete	`bo.yura.app` (cambiado desde `com.example.yura_app`)
Stack	Kotlin 2.2.10, Jetpack Compose, Retrofit, DataStore, minSdk 26
Pantallas	Ingreso y «Mis hijos» (solo `id` y `nombre`)
Pruebas	2 unitarias con servidor simulado, pasan
Git	Repositorio propio, rama `main` , 1 commit; llaves de firma excluidas
Compilación	`assembleDebug` correcto
No verificado	Ejecución contra la web real y en un dispositivo
API de publicación	Pendiente: `https://CAMBIAR-POR-DOMINIO/`

Web (`C:\xampp\htdocs\yura-platform`)

- API existente: `/api/login` , `/api/logout` , `/api/user` , `/api/dashboard` , `/api/students` , `/api/users` , rutas del robot `/api/v1/robot/...` (contrato v1.2 con el cerebro).
- MySQL real desactualizado: le faltan `robot_activities` y `robot_activity_attempts` (hallazgo de la auditoría anterior).

3. Fallas encontradas en la web

#	Falla	Ubicación	Consecuencia	Fase
F1	`/api/dashboard` y `/api/students` para familias no filtran por `student_family_members.active`; la web sí lo hace	`Api/DashboardController`, `Api/StudentController@index`	Un familiar dado de baja sigue viendo al niño por la API	A
F2	La API devuelve el modelo `Student` completo con `padre` y `profesor` anidados	mismos controladores	Expone datos de contacto de más	B
F3	Los tokens nunca vencen (`expiration => null`)	`config/sanctum.php`	Un celular perdido da acceso indefinido	A
F4	`POST /api/v1/robot/activity` no verifica el rol	`routes/api.php`	Cualquier usuario autenticado puede enviar notas	A
F5	`/api/login` sin límite de intentos	`AuthController`	Se pueden probar contraseñas sin freno	A
F6	Los tokens de usuarios suspendidos siguen válidos	rutas `auth:sanctum`	La suspensión solo bloquea el ingreso nuevo	A
F7	MySQL sin las tablas del robot	base de datos	Rutas del robot responden 500	Aparte (requiere aprobación)

Verificado leyendo el código. Todavía no se reprodujo cada falla con una prueba; eso se hace al inicio de la Fase A.

4. Diseño propuesto

API nueva para la app (la existente no se rompe):

- `GET /api/v1/app/me`
- `GET /api/v1/app/children`
- `GET /api/v1/app/children/{student}/summary`

Reglas:

- Respuestas con *API Resources*: solo los campos que la app necesita.
- Autorización con `User::canViewStudent` y vínculo familiar activo; un estudiante ajeno responde 403.
- El resumen reutiliza el cálculo de `DashboardController` (promedios por trimestre y materia, asistencia, retos del robot) movido a un servicio compartido, para que web y app muestren los mismos números.

- No se modifican el contrato del robot v1.2 ni las rutas actuales de la web, salvo F1, F4 y F6.

App: lista de hijos con resumen, detalle (trimestres, materias, asistencia, retos del robot), estados vacío/error/sin red y manejo de sesión vencida.

5. Plan por fases

Fase	Contenido	Responsable
A	Pruebas Pest que reproducen F1, F3–F6; luego las correcciones	Claude, en `yura-platform`
B	Servicio compartido, Resources, rutas `v1/app` y pruebas por rol (padre propio, ajeno, vínculo inactivo, estudiante, profesor)	Claude
C	Segunda opinión de seguridad sobre A y B, solo lectura, sin datos reales	Codex
D	Pantallas y ViewModels del detalle, con pruebas	Subagente en `yuraapp`, sobre el contrato de B
E	Prueba real con servidor local, capturas y registro de cambios	Claude

B y D corren en paralelo cuando el JSON esté fijado (máximo 2 subagentes a la vez).

6. Reglas de ejecución

- Suite Pest completa (52 pruebas) antes y después de cada fase; nada que pasaba puede quedar fallando.
- Commits pequeños por fase, con las pruebas pasando.
- Rama de la web: `feature/api-app-v1` (no se trabaja directo sobre `develop`).
- **`php artisan migrate` en MySQL no se ejecuta sin aprobación aparte.**
- Las decisiones no técnicas se consultan antes de aplicarlas.
- No se envían datos de estudiantes a servicios externos.

7. Decisiones tomadas (valores por defecto del plan aprobado)

Tema	Valor	Cambiable
Vigencia del token en la app	30 días	Sí
Notas que ve el padre	Solo `status = completed` (igual que el panel web)	Sí
Rama de la web	`feature/api-app-v1`	Sí

8. Riesgos y límites

- La app nunca se ha probado en un dispositivo ni contra la web real.
- Para publicar en Play Store hace falta un servidor público con dominio y HTTPS; hoy solo existe la LAN.
- Cambiar la vigencia de los tokens cierra las sesiones actuales de la API.

- La tesis dice que la app Kotlin no existe. Mientras no esté terminada y probada, debe figurar como trabajo futuro.

9. Criterio de «terminado» para esta versión

1. F1–F6 corregidas, cada una con una prueba que fallaba antes y pasa ahora.
2. Suite Pest completa en verde.
3. Las 3 rutas `v1/app` con pruebas por rol.
4. La app compila, sus pruebas pasan y se probó el ingreso y el detalle contra la web local con un padre de prueba.
5. Este reporte actualizado a la v1.1 con lo realmente hecho y lo que falló.